



Informe - Nonix

Diseño e Implementación de un Lenguaje

1° Cuatrimestre de 2025

Equipo:

Nombre	Apellido	Legajo	E-mail
Juan Pablo	Birsa	64.500	jbirsa@itba.edu.ar
Conrado	Hillar	64.633	chillar@itba.edu.ar
Ignacio	Maruottolo	64.611	imaruottoloquioga@itba.edu.ar
Ezequiel	Testoni	64.709	etestoni@itba.edu.ar

Tabla de contenidos

1. Introducción	2
2. Modelo computacional	2
2.1. Dominio	2
2.2. Lenguaje	2
3. Implementación	5
3.1. Frontend	5
3.1.1. Análisis léxico	5
3.1.2. Análisis sintáctico	7
3.2. Backend	10
3.2.1. Análisis semántico	10
3.2.2. Generación de código	13
3.3. Dificultades encontradas	16
4. Futuras extensiones	17
5. Conclusiones	17
6. Referencias	18

1. Introducción

En este informe se detalla la implementación de un DSL (Domain Specific Language) llamado NONIX, junto con su compilador, que permite la definición y análisis de variables, fórmulas y conectivos del lenguaje de la lógica proposicional.

2. Modelo computacional

2.1. Dominio

El dominio del lenguaje NONIX se fundamenta en la lógica proposicional. Para el desarrollo del proyecto, la información fue principalmente obtenida de apuntes de la materia Lógica Computacional (93.35) de la carrera de Ingeniería en Informática del Instituto Tecnológico de Buenos Aires, más específicamente, de *Las Sagradas Escrituras de la Lógica Computacional - 93.35*¹.

Si bien el proyecto no abarca todos los conceptos que puede expresar la lógica proposicional, sí opera sobre muchos de sus elementos principales: variables, fórmulas, valuaciones, conectivos y tablas de verdad. Además, NONIX extiende algunas funcionalidades más allá de la lógica proposicional formal, permitiendo, por ejemplo, definir nuevos operadores y utilizar wildcards en lugar de los valores booleanos *true* y *false*. Estas extensiones le dan al usuario mayor flexibilidad y poder de expresión, facilitando el modelado de situaciones que no serían posibles dentro de la lógica clásica.

2.2. Lenguaje

El lenguaje Nonix permite siete tipos de sentencias: cinco son definiciones y las dos restantes son las funciones ejecutables *evaluate* y *adequate*.

a. Definición de variables.

```
// Definición de una variable
define variable x;

// Definición de múltiples variables
define variable p, q, s, t;
```

Nótese que no es posible asignar un valor de verdad a la variable dentro de la definición, simplemente se declara su existencia dentro del programa.

¹ M. Zahnd, “[Las Sagradas Escrituras de Lógica Computacional - 93.35](#)”: Apuntes de la materia Lógica Computacional (93.35) según se dictó durante los años 2020, 2021 y el primer cuatrimestre del 2022 en el Instituto Tecnológico de Buenos Aires (ITBA)*, GitHub repository, 2024.

b. Definición de fórmulas. Antes de detallar la sintaxis de la definición de una fórmula, es necesario hablar de las expresiones porque, de hecho, una fórmula es una expresión. Sea α una secuencia de símbolos, si α cumple alguno de los siguientes casos:

- $\alpha = x$
- $\alpha = \{myForm\}$
- $\alpha = myOp(x, y)$
- $\alpha = (\beta \ \& \ \delta)$
- $\alpha = (\beta \ | \ \delta)$
- $\alpha = (\beta \Rightarrow \delta)$
- $\alpha = (\beta \Leftrightarrow \delta)$
- $\alpha = !\beta$

(donde x e y son variables definidas previamente, $myForm$ es el identificador de una fórmula definida previamente, $myOp$ es el identificador de un operador definido previamente, β y δ son expresiones), entonces α es una expresión válida.

Con esto, la definición de una fórmula sigue la estructura que se muestra a continuación:

```
// Definición de una fórmula
define formula myForm = _expression_;
```

donde `_expression_` debe ser una expresión válida según las reglas recién descritas.

c. Definición de valuaciones.

```
// Definición de una valuación
define valuation myVal = {p = true, q = false, ...};
```

donde p , q y el resto de las variables a las que se esté asignando un valor de verdad deben haber sido definidas previamente.

d. Definición de operadores. La definición de un operador se hace a partir de una tabla de verdad. Entonces, resulta fundamental especificar en primer lugar cuál es la estructura de dicha tabla.

La tabla de verdad representa una función $\{boolean^N\} \rightarrow \{boolean\}$, es decir que mapea una tupla de booleanos a otro booleano. Cada entrada de la tabla refleja este mapeo para una serie particular de booleanos y el lenguaje NONIX permite realizar esto a través de dos formas sintácticas diferentes:

- Caso 1: $(boolean1, boolean2, \dots, booleanN) \rightarrow mapBoolean;$
- Caso 2: $mapBoolean otherwise;$

El Caso 1 representa la definición tradicional de la función para una tupla en particular del dominio. La única aclaración a hacer es que, en este caso, cada elemento *booleanK* de la serie puede tomar valores *true*, *false* o wildcard (?). Este es un agregado que le hace NONIX a la lógica proposicional, con el fin de facilitar la tarea al programador. La idea es la siguiente: si dos o más tuplas de booleanos se mapean al mismo valor de verdad independientemente del valor que tome su k-ésimo booleano, luego ese conjunto se puede resumir en una sola tupla que contenga el wildcard en el k-ésimo lugar. Por definición de función, la misma tupla no puede aparecer mapeada a dos booleanos distintos en la misma tabla, y sobre este punto hay que tener especial cuidado con el uso del wildcard.

El Caso 2 representa la definición por default para un conjunto de tuplas del dominio. Esto también busca simplificar la tarea del programador, ya que en lugar de obligarlo a definir la función para cada tupla del dominio, le permite utilizar este valor por defecto que representa a un conjunto de tuplas que devuelven el mismo booleano. Al computar un operador que contenga una entrada de este tipo, en caso de que la serie de parámetros recibida no matchee con ninguna entrada más específica (del Caso 1) se devolverá este valor por defecto. Por supuesto que no puede haber más de una entrada de este tipo en la misma tabla.

A partir de esta estructura, la definición de un operador se realiza de la siguiente forma:

```
// Definición de un operador
define operator myOp(x, y, z) = {
  _truth_table_entries_
};
```

donde *x*, *y*, *z* son variables genéricas (simplemente indican la cantidad de parámetros que recibe el operador) y *_truth_table_entries_* es la lista de entradas de la tabla de verdad, encolumnadas una debajo de la otra.

e. Definición de opsets. Un opset es, sencillamente, una lista de operadores. Luego, la definición de un opset tiene la siguiente sintaxis:

```
// Definición de un opset
define opset myOpset = {myOp1, ..., myOpN, &, |, ... };
```

donde *myOp1*, ..., *myOpN* son los identificadores de operadores previamente definidos y *&*, *|*, ... son los conectivos tradicionales de la lógica proposicional. Adviértase que la notación utilizada no implica la obligatoriedad de la existencia de todos estos elementos. La lista de

operadores puede contener solo operadores predefinidos, solo conectivos tradicionales o una combinación de las dos clases. No puede estar vacía.

f. Función *evaluate*. La función *evaluate* realiza el cómputo de la expresión asociada a una función a partir de una valuación definida, como mínimo, para cada variable que forme parte de dicha expresión. La sintaxis de la sentencia es la siguiente:

```
// Función evaluate
evaluate(myForm, myVal);
```

donde *myForm* es el identificador de la función a evaluar (definida previamente) y *myVal* es el identificador de una valuación (también definida previamente) que determina el valor de verdad de las variables involucradas en la expresión a evaluar.

g. Función *adequate*. La función *adequate*, como indica su nombre, verifica si el opset recibido es o no un conjunto adecuado de conectivos. La adecuación es una propiedad particular de los conjuntos de conectivos de la lógica proposicional. Más adelante, en la sección **3.2.2. Generación de Código** de este informe, se darán más detalles sobre cómo se lleva a cabo dicho cómputo. La sentencia se estructura de esta manera:

```
// Función evaluate
adequate(myOpset);
```

donde *myOpset* es el identificador de un opset definido previamente.

3. Implementación

3.1. Frontend

El Frontend del parser desarrollado se encarga de dos tareas fundamentales: el análisis léxico y sintáctico del programa recibido como entrada.

3.1.1. Análisis léxico

La primera etapa es la del análisis léxico, en la cual se reconocen y procesan las cadenas de caracteres ingresadas de acuerdo con las reglas y acciones de Flex. En este caso, los patrones de caracteres reconocibles por el parser están definidos en el archivo *FlexPatterns.l*. Allí se pueden distinguir distintos tipos de lexemas: las palabras reservadas, como *define*, *evaluate*, *otherwise*, entre otras; los caracteres asociados a conectivos (&, |, =>, <=>, !); los identificadores de estructuras — ya sean variables, fórmulas, nuevos operadores, valuaciones u opsets —, que deben ser cadenas formadas por una o más letras, números o ‘_’

y no pueden comenzar con un número; y otros símbolos, como '=', '\$', '?', '{', '->', etc., que también forman parte del alfabeto.

Ante el reconocimiento de cada uno de estos lexemas, el parser activa la acción de Flex que lleva asociada y que permite configurar el token que luego se utilizará en la próxima etapa del análisis. En el archivo *FlexActions.c* se definen cuatro acciones de Flex que manejan estos tokens. La primera es *OnlyTokenLexemeAction* que, como indica su nombre, devuelve el token tal cual lo recibe, sin realizar ninguna manipulación. Este método se asocia a lexemas cuyo valor semántico no resulta relevante al resto del análisis y posterior cómputo del programa, como las palabras reservadas y los símbolos que no representan conectivos. Otra de las acciones es *SemanticValueLexemeAction* que, antes de devolver el token, asigna a su campo *semanticValue* un puntero al duplicado del lexema leído. Aquí caben dos aclaraciones:

Por un lado, como se sabe, el campo *semanticValue* de cada token representa la unión de una serie de distintos tipos de datos. En el caso de este parser, para los terminales, el *semanticValue* puede guardar un *keywordOrSymbol* (const char *) o un *truth_value* (boolean). La función *SemanticValueLexemeAction*, en particular, se encarga de guardar los lexemas de tipo *keywordOrSymbol*, es decir que está asociada tanto a los identificadores como a los caracteres representativos de conectivos, cuyo valor semántico sí tiene importancia en etapas posteriores del procesamiento del programa.

La segunda aclaración, asumiendo el riesgo de caer en imprecisiones, busca explicar resumidamente el motivo de por qué se duplica el lexema antes de guardarse en el *semanticValue*. Flex emplea un buffer interno para armar estos tokens que luego le pasa a Bison. Ese buffer es reutilizado por Flex en cada llamada, por lo que cualquier puntero directo a él corre el riesgo de volverse inválido después de leer el siguiente token. Es por esto que, si se necesita que el valor del lexema persista más allá de la acción léxica, es necesario duplicar su contenido y almacenar esa copia.

Habiendo hecho ambas aclaraciones, falta mencionar la tercera acción definida. Previamente se dijo que eran cuatro, pero en concreto las dos acciones restantes tienen el mismo comportamiento: guardan en el *semanticValue* del token el valor correspondiente a *truth_value*. Se decidió implementar una función para cada tipo de booleano, *TrueSemanticValueLexemeAction* y *FalseSemanticValueLexemeAction*, con el fin de evitar realizar un segundo parseo sobre el mismo lexema. Obviamente, estas acciones están asociadas a las palabras reservadas *true* y *false*, respectivamente.

Finalmente, queda destacar el caso particular de los patrones */** y **/*, que si bien no generan un token como los demás lexemas, sí activan una acción de Flex que permite cambiar el contexto de análisis entre el estándar y el *MULTILINE_COMMENT*. Dentro del contexto *MULTILINE_COMMENT*, el resto de las acciones descritas anteriormente no tiene validez justamente porque se trata de un comentario.

Cualquier otro patrón de caracteres que no haya sido detallado en esta sección se ignora, como en el caso de los espacios, o directamente se marca como desconocido.

3.1.2. Análisis sintáctico

En cuanto a la etapa de análisis sintáctico, el conjunto de producciones de la gramática definida para el lenguaje Nonix es el siguiente:

```
program: program statement SEMICOLON
      | statement SEMICOLON
      ;

statement: defineVariable
      | defineFormula
      | defineValuation
      | defineOperator
      | defineOpset
      | evaluateStatement
      | adequateStatement
      ;

defineVariable: DEFINE VARIABLE variableList
      ;

defineFormula: DEFINE FORMULA IDENTIFIER EQUALS expression
      ;

defineValuation:
      | DEFINE VALUATION IDENTIFIER EQUALS OPEN_BRACE valuationList CLOSE_BRACE
      ;

variableList: IDENTIFIER
      | variableList COMMA IDENTIFIER
      ;

valuationList: valuation
      | valuationList COMMA valuation
      ;

valuation: IDENTIFIER EQUALS truthValue
      ;

defineOperator:
      | DEFINE OPERATOR customOperator EQUALS OPEN_BRACE truthTable CLOSE_BRACE
      ;

customOperator: IDENTIFIER OPEN_PARENTHESIS variableList CLOSE_PARENTHESIS
      ;

truthTable: truthTableEntry
      | truthTable truthTableEntry
      ;

truthTableEntry: truthValue OTHERWISE SEMICOLON
      | OPEN_PARENTHESIS truthValueList CLOSE_PARENTHESIS ARROW truthValue SEMICOLON
      ;

truthValueList: truthValueOrWildcard
      | truthValueList COMMA truthValueOrWildcard
```



```

;

truthValueOrWildcard:
| WILDCARD
| truthValue
;

truthValue:
| TRUE
| FALSE
;

defineOpset: DEFINE OPSET IDENTIFIER EQUALS OPEN_BRACE opsetList CLOSE_BRACE
;

opsetList: opsetList COMMA IDENTIFIER
| opsetList COMMA AND
| opsetList COMMA OR
| opsetList COMMA THEN
| opsetList COMMA IFF
| opsetList COMMA NOT
| IDENTIFIER
| AND
| OR
| THEN
| IFF
| NOT
;

evaluateStatement:
| EVALUATE OPEN_PARENTHESIS IDENTIFIER COMMA IDENTIFIER CLOSE_PARENTHESIS
;

adequateStatement: ADEQUATE OPEN_PARENTHESIS IDENTIFIER CLOSE_PARENTHESIS
;

expression: binaryExpression
| customExpression
| notExpression
| IDENTIFIER
;

binaryExpression:
| OPEN_PARENTHESIS expression[left] AND[op] expression[right] CLOSE_PARENTHESIS
| OPEN_PARENTHESIS expression[left] OR[op] expression[right] CLOSE_PARENTHESIS
| OPEN_PARENTHESIS expression[left] THEN[op] expression[right] CLOSE_PARENTHESIS
| OPEN_PARENTHESIS expression[left] IFF[op] expression[right] CLOSE_PARENTHESIS
;

customExpression: DOLLAR OPEN_BRACE IDENTIFIER CLOSE_BRACE
| customOperator
;

notExpression: NOT expression
;

```

Con respecto a la gramática definida, no hay mucho que explicar más allá de lo detallado en la sección **2.2. Lenguaje**.

Cada producción de la gramática lleva asociada una acción de Bison que es responsable fundamentalmente de construir su nodo correspondiente en el Abstract Syntax Tree (AST). En muchas de estas acciones, además, se realizan chequeos semánticos, pero este tema se abordará en profundidad más adelante en la sección **3.2.1. Análisis semántico**. Un detalle a aclarar es que a los nodos del AST que guardan aquellos lexemas duplicados en la acción de Flex *SemanticValueLexemeAction* (ver sección **3.1.1 Análisis léxico**), se les pasa un nuevo puntero duplicado — un duplicado del duplicado. En este caso, la decisión no tiene que ver con la persistencia de los datos sino con mantener una independencia entre las estructuras, de modo que cada una pueda hacerse responsable de liberar correctamente sus recursos y evitar conflictos. De hecho, inmediatamente después de asignar el nuevo duplicado al nodo, la propia acción de Bison libera el duplicado original.

Sobre la construcción del AST y la estructura de los nodos, simplemente comentar algunos detalles en relación a cómo se manejan los casos de no-terminales que pueden derivar en más de una secuencia de símbolos distinta. Por ejemplo, el no-terminal *binaryExpression*:

```
binaryExpression:
    | OPEN_PARENTHESIS expression AND expression CLOSE_PARENTHESIS
    | OPEN_PARENTHESIS expression OR expression CLOSE_PARENTHESIS
    | OPEN_PARENTHESIS expression THEN expression CLOSE_PARENTHESIS
    | OPEN_PARENTHESIS expression IFF expression CLOSE_PARENTHESIS
    ;
```

O el no-terminal *truthTableEntry*:

```
truthTableEntry:
    | OPEN_PARENTHESIS truthValueList CLOSE_PARENTHESIS ARROW truthValue SEMICOLON
    | truthValue OTHERWISE SEMICOLON
    ;
```

En estos casos, se definieron algunos enums útiles como:

```
// AbstractSyntaxTree.h
...
enum BinaryOperatorType {
    BINOP_AND,
    BINOP_OR,
    BINOP_THEN,
```

```

        BINOP_IFF
    };

    ...
enum TruthTableEntryType {
    TRUTH_VALUE_LIST,
    OTHERWISE_ENTRY
};

```

Con esta implementación, una estructura típica de un nodo del AST resulta ser: el tipo de secuencia a la que deriva el nodo (elemento del enum) y una unión de estructuras que representan cada variante posible. Para el caso de *truthTableEntry*, por ejemplo, se tiene:

```

// AbstractSyntaxTree.h
...
struct TruthTableEntry {
    union {
        struct {
            TruthValueList * truthValueList;
            TruthValue * mapValue; // Boolean to which the list is mapped.
        };
        TruthValue * otherwiseValue;
    };
    TruthTableEntryType type;
};

```

3.2. Backend

Para el Backend del parser, también se reconocen dos fases principales: una de análisis semántico y otra de generación de código.

3.2.1. Análisis semántico

El análisis semántico del programa se superpone en cierta medida con la etapa de análisis sintáctico porque gran parte del mismo se lleva a cabo en las propias acciones de Bison. Se tomó esta decisión fundamentalmente para evitar realizar recorridos innecesarios al AST.

El punto más importante de esta fase es el armado de la Tabla de Símbolos, que permite regularizar el uso de los identificadores y además guarda la información necesaria para luego poder realizar el chequeo de tipos. Con respecto a la estructura de la Tabla de Símbolos, se ha integrado al proyecto del código fuente de una implementación de hashmap

en C², denominada *uthash.h* por el autor (renombrada como *Hashmap.h* en el repositorio). Por otra parte, para cada entrada de la tabla se ha definido una estructura que mantiene un formato similar al de los nodos complejos del AST (ver sección 3.1.2 Análisis sintáctico):

```
// SymbolTable.h
...
typedef enum {
    SYMBOL_VARIABLE,
    SYMBOL_FORMULA,
    SYMBOL_VALUATION,
    SYMBOL_OPERATOR,
    SYMBOL_OPSET,
} SymbolType;

typedef struct SymbolEntry {
    const char *name;           // Nombre del identificador (Lexema)
    SymbolType type;            // Tipo del identificador
    UT_hash_handle hh;         // Manejo interno del hashmap (irrelevante)
    union {
        struct {
            int argc;
            TruthTable *truth_table;
        } operator_data;       // SYMBOL_OPERATOR
        boolean truth_value;    // SYMBOL_VARIABLE
        Expression *expression_node; // SYMBOL_FORMULA
        ValuationList *valuation_list; // SYMBOL_VALUATION
        OpsetList *opset_list;  // SYMBOL_OPSET
    } data;
} SymbolEntry;
```

Véase que cada tipo de identificador se asocia a una estructura determinada, que luego será utilizada para realizar el chequeo de tipos o bien la generación de código propiamente dicha. Nuevamente vale aclarar que, al igual que sucede con los nodos del AST, el nombre del identificador, es decir, el lexema que se almacena en la Tabla de Símbolos también es un segundo duplicado del puntero original. Y además cabe remarcar que el nombre del identificador se guarda junto con el resto de las estructuras porque la implementación de *uthash.h* requiere que la clave del mapa sea un campo de la entrada. Esto último se menciona para evitar confusiones con respecto al manejo del par (key, value) propio de las estructuras de hashmap implementadas por otros lenguajes, como Java.

² Implementación de hashmap en C, *uthash.h*, desarrollada por Troy D. Hanson:
<https://raw.githubusercontent.com/troydhanson/uthash/master/src/uthash.h>

En el archivo *SymbolTable.c*, se definen funciones para insertar, buscar y actualizar entradas de la Tabla, así como para liberar los recursos una vez finalizado el parsing. En cada acción de Bison asociada a una sentencia de tipo definición, se inserta en la Tabla de Símbolos el identificador y su estructura correspondiente, validando antes que dicho identificador no haya sido definido previamente. En las demás acciones de Bison que manejan identificadores, simplemente se valida que dichos identificadores sí hayan sido declarados anteriormente en alguna sentencia. Hay un solo caso que se maneja de manera especial, que es el del *CustomOperator*. Como se explicó en la sección **2.2. Lenguaje**, el operador se define de manera genérica. Es decir, cuando se escribe:

```
// myProgam.nonix
...
define operator myOp(x, y, z) = {
    (true, false, true) -> true;
    false otherwise;
};
```

las variables *x*, *y*, *z* son genéricas. Por lo tanto, no deben estar declaradas previamente ni se insertan en la Tabla de Símbolos a partir de esta definición.

El otro punto clave de esta fase es el chequeo de tipos. En el archivo *TypeChecking.c*, se definen todas las funciones referidas a este procedimiento. Las validaciones implementadas incluyen no solo chequeos directos sobre el tipo de los identificadores, sino también la verificación de otras condiciones que garantizan la validez semántica del programa. Por ejemplo, dada la definición de un operador que recibe *N* parámetros, se verifica:

- Que la cantidad de columnas en cada entrada de la *TruthTable* asociada sea igual a *N*.
- Que la cantidad de entradas de la tabla no supere 2^N .
- Que no se repita la misma tupla más de una vez (independientemente del booleano al que se mapea).
- Que estén todas las tuplas posibles representadas. Es decir, que la función esté bien definida para todo su dominio.
- Y que si se utiliza este operador en otra sentencia, reciba los *N* parámetros.

Para validar el tercer punto, se decidió utilizar otro hashmap (también tomando la implementación *uthash.h*) cuyas claves son las codificaciones de cada tupla de la tabla en binario. Por ejemplo, para una entrada (true, false) -> *booleanX*; la clave con la que se inserta en el mapa resulta ser: $10b = 2$. Para la entrada (false, true) -> *booleanY*; la codificación correspondiente es: $01b = 1$. Y así sucesivamente, para cada tupla de la tabla. Nótese que la codificación es independiente del valor al que se mapea la tupla, como se explicita en el ítem listado. Dada una tupla que incluya wildcards — como (true, false, ?) -> *booleanZ*; —, se reemplazan dichos wildcards por cada valor de verdad que podrían tomar y se insertan todas las tuplas resultantes en la tabla — en este caso se insertarán las claves (true, false, true) =

$101b = 5$ y $(true, false, false) = 100b = 4$. Por otro lado, esta implementación permite verificar fácilmente el cuarto punto porque, luego de insertar todas las claves en el mapa, si su tamaño es distinto a 2^N quiere decir que no se ha definido el mapeo para todas las tuplas posibles. Esta validación (la del tercer y cuarto punto) se aplica únicamente a las tablas donde no existe una tupla de tipo *OTHERWISE_ENTRY*.

3.2.2. Generación de código

En esta fase final de la compilación del programa de entrada se llevan a cabo dos procesos principales: el cómputo del programa y la generación de la salida.

Tal como se describe en la sección **2.2. Lenguaje**, Nonix ofrece dos funciones ejecutables que son *evaluate* y *adequate*. Luego, el cómputo del programa consiste exclusivamente en resolver estas dos funciones siguiendo los lineamientos teóricos de la lógica proposicional. Para resolver *evaluate*, en primera instancia se actualizan en la Tabla de Símbolos los valores de verdad de las variables involucradas en la fórmula recibida como parámetro, a partir de la valuación especificada. Con estos valores, se resuelve recursivamente el valor de verdad de la expresión asociada a la fórmula. En el proceso es probable que se computen diversas estructuras. Para las expresiones binarias y las expresiones negadas se aplican los condicionales correspondientes. Si hubiera una fórmula predefinida, se resuelve primero siguiendo este mismo procedimiento. Y el caso más complejo es el del *CustomOperator* porque hay que analizar su *TruthTable* asociada. Para resolverlo se sigue el siguiente algoritmo (revítese la estructura de la *TruthTable* desarrollada en la sección **2.2. Lenguaje**):

1. Se buscan las variables recibidas como parámetros en la Tabla de Símbolos y se mapean sus valores de verdad a un array temporal.
2. Se crea una estructura *ComputationResult* y se comienza a iterar por las entradas de la *TruthTable*. Por cada una, hay dos posibilidades:

2.1 Si la entrada es del tipo *TRUTH_VALUE_LIST*, se recorre la lista de valores de verdad de la entrada y se los compara uno a uno con los del array temporal. Nótese que en cada caso, el valor de verdad de la entrada podría ser un booleano (*true* o *false*) o un wildcard (?). Siguiendo la lógica de la sintaxis definida para Nonix, ante un wildcard (?) en la entrada, la comparación siempre devuelve *true*. Si todos los valores coinciden, se retorna, marcando el *ComputationResult* como exitoso y actualizando su valor con el *mapValue* correspondiente a la entrada. Si no, se sigue con la próxima entrada.

2.2 Si la entrada es del tipo *OTHERWISE_ENTRY*, se guarda el valor de verdad asociado a la entrada en el *ComputationResult* y se marca como exitoso, pero no se retorna, ya que hay que esperar por una posible coincidencia más específica en el resto de las entradas.

3. Finalizado el ciclo, si el *ComputationResult* quedó marcado como exitoso, esto implica que no hubo ninguna entrada más específica que coincidiera y se retorna el valor del *otherwise*. En caso contrario, los parámetros recibidos no coinciden con ninguna entrada de la *TruthTable*, lo cual revela un error en la definición del operador o en la lista de variables recibida (si el parser funciona correctamente, este error debería ser detectado en etapas anteriores, ya sea de análisis sintáctico o semántico).

Para resolver *adequate* se utiliza el *Teorema de Completitud Funcional de Post*³. Este plantea que un conjunto de conectivos es funcionalmente completo (adecuado) si y sólo si para cada una de las siguientes cinco clases de funciones, algún conectivo del conjunto no pertenece a ella.

1. Funciones **T-preservantes**: funciones que devuelven *true* cuando todos los argumentos son *true*. Es decir, $f(T, T, \dots T) = T$
2. Funciones **F-preservantes**: funciones que devuelven *false* cuando todos los argumentos son *false*. Es decir, $f(F, F, \dots F) = F$
3. Funciones **monotónicas**: funciones que no disminuyen su valor al incrementar las variables de entrada. Particularmente, las funciones monotónicas booleanas son aquellas en las que cambiar de *false* a *true* en alguna variable nunca hace que la salida cambie de *true* a *false*. Es decir, para todo par de valuaciones $\langle x_1, \dots x_n \rangle$ y $\langle y_1, \dots y_n \rangle$, si $\langle x_1, \dots x_n \rangle \leq \langle y_1, \dots y_n \rangle$ entonces $f(x_1, \dots x_n) \leq f(y_1, \dots y_n)$. Se asume que $F < T$, y además que una valuación es mayor que otra si la cantidad de variables con valor T es mayor.
4. Funciones **auto-duales**: funciones en las que negar todas las variables de entrada equivale a negar la salida. Es decir, para toda valuación $\langle x_1, \dots x_n \rangle$, $f(x_1, \dots x_n) = \neg f(\neg x_1, \dots \neg x_n)$
5. Funciones de **conteo** (counting): funciones en las que la salida depende únicamente del número de entradas verdaderas (por ejemplo, XOR). Visto de otra manera, la p

Con este marco teórico, para verificar que un opset es adecuado es suficiente con validar que para cada una de las cinco propiedades, hay por lo menos un operador del conjunto que no la cumple. Por el contrario, si se encuentra una propiedad que cumplen todos los operadores, se puede afirmar que el conjunto no es adecuado. Para más detalle acerca del Teorema, en la sección **6. Referencias** se encuentra el paper del cual se realizó la investigación.

Teniendo en cuenta la posibilidad de que exista más de una sentencia *evaluate* o *adequate* en un programa, los resultados correspondientes a cada llamada a una de dichas funciones se guardan en una lista. De este modo, al finalizar el cómputo del programa completo se devuelven todos y en el orden adecuado.

³ F. J. Pelletier and N. M. Martin, “[Post’s Functional Completeness Theorem](#)”, *Notre Dame Journal of Formal Logic*, vol. 31, no. 2, pp. 462–475, Spring 1990.

En el archivo *Generator.C*, se encuentran todas las funciones referidas a la generación de la salida. En este caso, se ha decidido mostrar por stdout únicamente los resultados obtenidos de las sentencias de tipo *evaluate* o *adequate*. En paralelo, se crea un archivo *output.tex*, donde se genera el código en LaTeX de todo el programa de entrada y se incluyen además, en una sección aparte, los resultados. Por ejemplo, para el siguiente programa de entrada:

```
// myProg.nonix
define variable p, q, r, s;
define valuation myVal = {p = true, q = false, r = false, s = false};
define formula myForm = ((p | (!q & r)) & ((r | !p) | !(s & q)));
evaluate(myForm, myVal);
```

El código generado en el archivo *output.txt* es:

```
// output.tex
\documentclass{article}
\usepackage[utf8]{inputenc}
\usepackage[T1]{fontenc}
\usepackage{amsmath}
\usepackage{microtype}
\usepackage{listings}
\usepackage{xcolor}
\usepackage{tcolorbox}
\date{}
\definecolor{codegray}{gray}{0.95}
\lstdefinestyle{mystyle}{backgroundcolor=\color{codegray},basicstyle=\ttfamily\small,frame=single,columns=fullflexible,keepspaces=true}
\begin{document}
\section*{Código generado}
\begin{lstlisting}[style=mystyle]
define variable p, q, r, s;
define valuation myVal = {p = true, q = false, r = false, s = false};
define formula myForm = ((p | (!q & r)) & ((r | !p) | !(s & q)));
evaluate(myForm, myVal);
\end{lstlisting}

\section*{Resultado}
\begin{tcolorbox}[colback=blue!5!white, colframe=blue!75!black, title=Resultado de evaluación]
evaluate(myForm, myVal) = true\end{tcolorbox}
\end{document}
```

que en un visualizador de LaTeX se ve de esta manera:

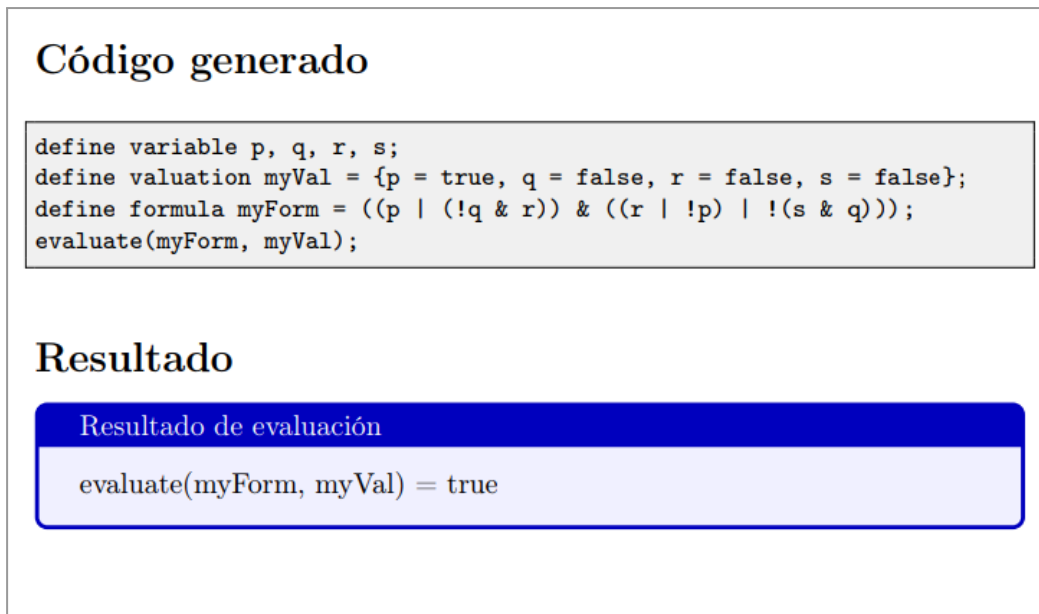


Figura 1 - Salida generada por un visualizador de LaTeX para el programa *myProg.nonix*

3.3. Dificultades encontradas

La principal dificultad encontrada durante la realización del proyecto tiene que ver con los destructores de Bison y los memory leaks. Cuando el proceso de parsing falla, ya sea porque el programa es inválido desde el punto de vista sintáctico o semántico o bien por un error interno del compilador, la ejecución se corta de manera abrupta y queda memoria asignada de forma dinámica sin liberar (memory leaks). Para solucionar este problema, se utilizan los destructores de Bison, que permiten ejecutar las funciones de liberación de memoria frente a un error imprevisto. Sin embargo, esta solución genera otro inconveniente: cuando hay destructores definidos, al finalizar el parsing Bison los activa independientemente de si el proceso resultó exitoso o no, con lo cual se pierde toda la estructura del AST antes de llegar a computar el programa. Esta situación imposibilita realizar una programación defensiva y cubrir todos los casos, ya que obliga a elegir uno de dos caminos: o se programa sin destructores y, ante un caso de error, quedan los memory leaks; o se definen los destructores de Bison para cubrirse de los memory leaks, pero se llega a un heap-use-after-free inevitable cuando se intenta computar el programa utilizando la estructura del AST que ya fue liberada antes de tiempo.

Un caso aparte es el de los lexemas duplicados. Como se detalló en varias secciones de este informe, cuando se trabaja con identificadores, Flex guarda en los tokens correspondientes un puntero duplicado del lexema original. A medida que Flex los genera, Bison va acumulando dichos tokens en una pila interna, de modo que, ante un error inesperado en la etapa de compilación, es probable que hayan quedado punteros duplicados sin liberar. Para solucionar esto, Bison también ofrece la posibilidad de asignar destructores a

los tokens que representan símbolos terminales de la gramática. Con esto, se debería poder solucionar esta falla. Pero, implementando estos destructores, el *sanitizer* informa que sobre estos punteros se realiza un double-free, es decir, que se está intentado liberar dos veces la misma zona de memoria. Esto se testeó exhaustivamente y se llegó a la conclusión de que este no es un problema de interferencia entre la tarea del destructor y de las funciones de liberación implementadas en el parser, teniendo en cuenta además que tanto el AST como la Tabla de Símbolos manejan sus propios punteros por decisión de diseño.

Otra dificultad inesperada fue la de implementar algorítmicamente la función *adequate*. De la manera que fue explicada en la materia Lógica Computacional, esta evaluación se realizaba de manera bastante mecánica, utilizando métodos como inducción global. Replicar esto de forma algorítmica representó un desafío.

Sin embargo, tras un poco de investigación, se halló un paper que detalla el Teorema de Completitud Funcional de Post. Como se explicó en la sección **3.2.2. Generación de código**, el paper plantea una forma mucho más imperativa de realizar el chequeo, a partir de 5 propiedades o “clases” a los que pueden pertenecer los conectivos del conjunto.

Con esta técnica, si bien no fue trivial de implementar, la lógica fue mucho más clara y se pudo abordar sin mayores complicaciones.

4. Futuras extensiones

En futuras versiones del lenguaje, se planea incorporar **cuantificadores lógicos** como $\forall$ (para todo) y $\exists$ (existe), lo que permitiría definir fórmulas más expresivas propias de la lógica de primer orden. A diferencia del sistema actual, donde toda fórmula necesita una valuación explícita para poder evaluarse, las fórmulas cuantificadas se evaluarían automáticamente sobre un dominio predefinido, sin requerir que el usuario indique el valor de cada variable. Esto habilitaría, por ejemplo, a escribir fórmulas como $\exists x. \forall y. (x \mid y)$, donde el compilador pueda analizar internamente todos los posibles valores para x e y , determinando si la fórmula es verdadera o no sin necesidad de una valuación. Esta extensión representaría un avance importante en la expresividad del lenguaje.

```
// myProg.nonix
define formula f =  $\exists x. \forall y. (x \mid y)$ ;
evaluate(f); // Resultado de evaluación: true
```

5. Conclusiones

El desarrollo de NONIX permitió poner en práctica conceptos vistos a lo largo de la cursada de la materia, mediante la implementación completa de un lenguaje específico de dominio basado en la lógica proposicional. A lo largo del proceso se abordaron todas las etapas de un compilador, desde el análisis léxico y sintáctico hasta el análisis semántico y la generación de código, aplicando diversas técnicas y estructuras para resolver todas las necesidades del proyecto.

Uno de los mayores logros fue la implementación algorítmica del verificador de adecuación de conectivos, basado en el Teorema de Completitud Funcional de Post. También la incorporación de algunas extensiones a la lógica formal, como la posibilidad de definir operadores personalizados y utilizar wildcards en las tablas de verdad, que le aportan al lenguaje una gran flexibilidad para el usuario.

Si bien se encontraron obstáculos técnicos, como el manejo correcto de la memoria frente a errores de parsing, estos fueron abordados con soluciones que priorizan la estabilidad del compilador.

En definitiva, en este trabajo se logró construir una herramienta poderosa para definir y evaluar expresiones y conectivos de la lógica proposicional de manera sencilla y flexible.

6. Referencias

- [1] M. Zahnd, “[Las Sagradas Escrituras de Lógica Computacional - 93.35](#)”: Apuntes de la materia Lógica Computacional (93.35) según se dictó durante los años 2020, 2021 y el primer cuatrimestre del 2022 en el Instituto Tecnológico de Buenos Aires (ITBA)*, GitHub repository, 2024.
- [2] Implementación de hashmap en C, *uthash.h*, desarrollada por Troy D. Hanson: <https://raw.githubusercontent.com/troydhanson/uthash/master/src/uthash.h>
- [3] F. J. Pelletier and N. M. Martin, “[Post’s Functional Completeness Theorem](#)”, *Notre Dame Journal of Formal Logic*, vol. 31, no. 2, pp. 462–475, Spring 1990.